Robotics and its Application (AI352) | Assignment 2

Python and Workspace Analysis

Student Name : *Naishadh Rana*
Roll. No : U23CS014

Part A: Get Familiar with jupyter notebook

```
# Program to check even and odd numbers using for loop and if condition
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    if num % 2 == 0:
        print(num, "is Even")
    else:
        print(num, "is Odd")
```

```
1 is Odd
2 is Even
3 is Odd
4 is Even
5 is Odd
```

Part B : Workspace Analysis of a 2R Planar Robot Arm

A **2R planar robot arm** consists of *two revolute joints* (θ_1, θ_2) and two rigid links (L_1, L_2). The end-effector's position is determined by as following equation for the links:

$x = L_1\cos(\theta_1) + L_2\cos(\theta_1 + \theta_2)$ $y = L_1\sin(\theta_1) + L_2\sin(\theta_1 + \theta_2)$

Workspace Visualization

Write a Python function compute_workspace(L1, L2, resolution) that:

- Sweeps through all possible (θ1, θ2) combinations
  - $\theta_1 \in [-\pi, \pi]$
  - $\theta_2 \in [-\pi, \pi]$
- Computes the end-effector positions (x, y) for each combination
- Returns arrays of x and y coordinates
- Plot the workspace using matplotlib as a scatter plot
- Label axes, add a title, and ensure the plot is axis-equal (no distortion)
- Explain how this workspace analysis is useful

```
import numpy as np
import matplotlib.pyplot as plt
```

```
def compute_workspace(L1, L2, resolution):
    theta_range = np.linspace(-np.pi, np.pi, resolution)
    theta1, theta2 = np.meshgrid(theta_range, theta_range)
    x = L1 * np.cos(theta1) + L2 * np.cos(theta1 + theta2)
    y = L1 * np.sin(theta1) + L2 * np.sin(theta1 + theta2)
    return x.flatten(), y.flatten()
```

```
def plot_workspace_grid(configs, resolution):
    fig, axes = plt.subplots(2, 2, figsize=(14, 12))
    axes = axes.flatten()

    for i, (L1, L2) in enumerate(configs):
        ax = axes[i]
        x, y = compute_workspace(L1, L2, resolution)
        distance = np.sqrt(x**2 + y**2) # distance for color mapping

        scatter = ax.scatter(x, y, s=2, c=distance, cmap='plasma', alpha=0.6)
        ax.set_title(f'Configuration {i+1}: L1={L1}, L2={L2}', fontsize=12, fontweight='bold')
        ax.set_xlabel('X Position (m)')
```

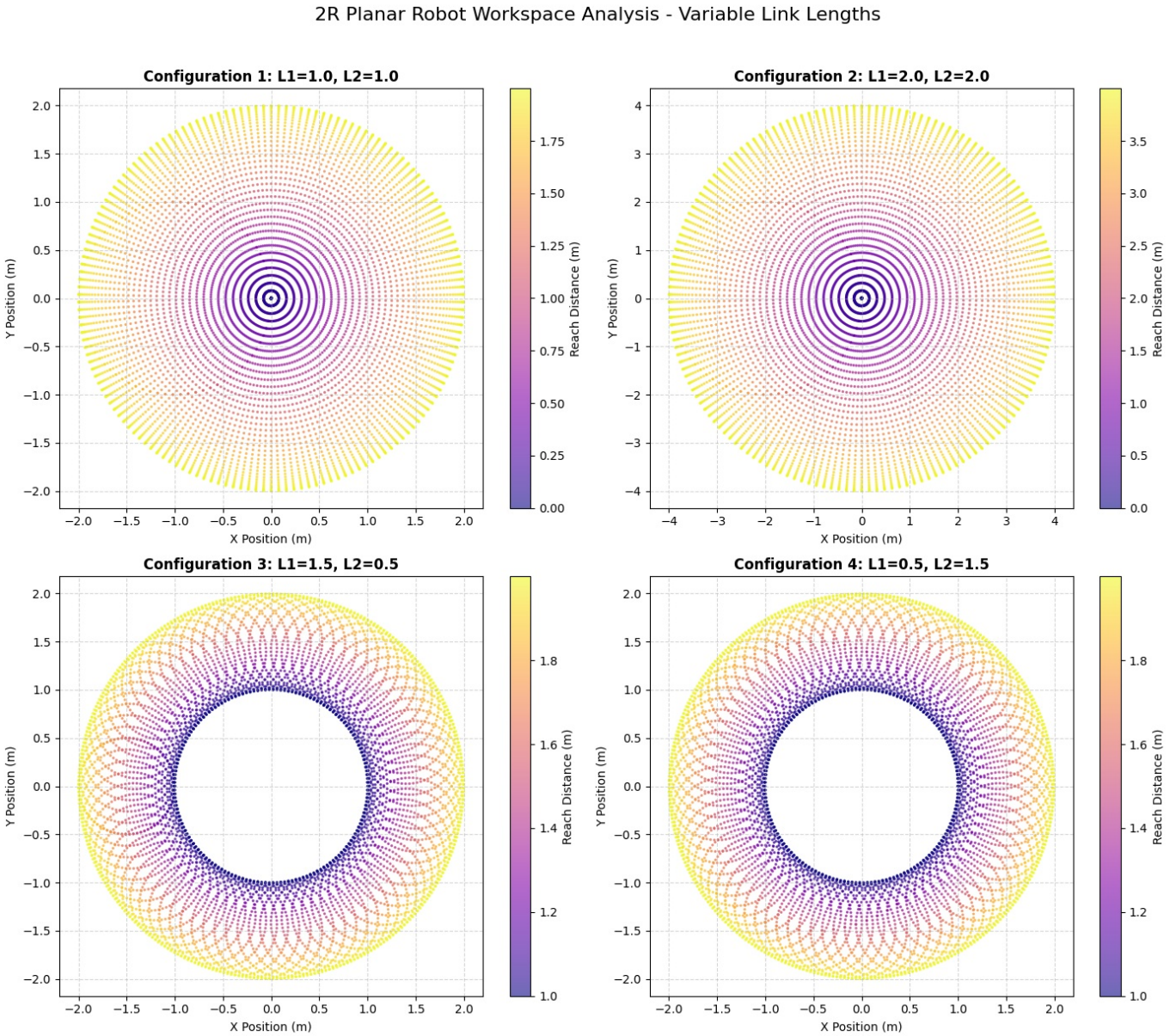
```
        ax.set_ylabel('Y Position (m)')
        ax.axis('equal')
        ax.grid(True, linestyle='--', alpha=0.5)

        cbar = plt.colorbar(scatter, ax=ax)
        cbar.set_label('Reach Distance (m)')

    plt.suptitle("2R Planar Robot Workspace Analysis - Variable Link Lengths", fontsize=16, y=1.02)
    plt.tight_layout()
    plt.show()
```

```
if __name__ == "__main__":
    link_configurations = [
        (1.0, 1.0),   # 1: Equal links
        (2.0, 2.0),   # 2: Scaled up version
        (1.5, 0.5),   # 3: Inner link longer
        (0.5, 1.5),   # 4: Outer link longer
    ]

    resolution = 80
    plot_workspace_grid(link_configurations, resolution)
```



> **Workspace Boundary Analysis**

- **Outer radius** when links align: $R_{outer} = L_1 + L_2$.
- **Inner radius** when links oppose: $R_{inner} = |L_1 - L_2|$.

Inside R_{inner} is unreachable; beyond R_{outer} is unreachable. We annotate both as circles centered at the origin.

> **Usefulness of Workspace Analysis:**

The workspace analysis of a robot is essential for several reasons:

1. **Task Feasibility:** Determines the **reachable workspace** (set of all points the end-effector can reach). Before programming a robot for a task (like pick-and-place or path planning), we must ensure all target points lie within this region.
2. **Robot Design:** Helps in selecting appropriate link lengths (L_1, L_2) to cover a desired working area (Example: paint spray job robots).
3. **Safety Zones:** Identifying the workspace boundary is crucial for safety, ensuring that the robot does not collide with obstacles or enter restricted zones outside its reach (Specially in cramped factories and hazardous environments).
4. **Dexterous Workspace:** It distinguishes between the reachable workspace (where the tip can reach) and the dexterous workspace (where the tip can reach with arbitrary orientation). For a 2R robot, we mainly look at position reachability.